1. Taking the classified intent from the ML team, what specific text processing techniques are you applying?

After receiving the predicted intent from the trained ML classifier (intent_classifier.pkl), the system applies the following NLP techniques:

a) Token Normalization

- Convert query to lowercase
- Remove noise characters
- Normalize spacing

b) spaCy-Based Linguistic Processing

- Tokenization
- POS tagging
- Lemmatization (where needed)

c) Dataset-Aware Rule-Based Matching

Instead of generic NER only, the system dynamically loads:

- Available product names
- Categories
- Regions
- Quarters
- Years

It then performs intelligent matching against real dataset values.

d) Regex-Based Time Extraction

Used for:

- Year detection (e.g., 2023)
- Quarter detection (Q1–Q4)
- Time references (last year, this year)

e) Intent-Based Routing

Once intent is identified:

Intent	Processing Logic
prediction	Aggregation on metric
compare	Multi-product groupby
trend	Time-series breakdown
summarise	KPI aggregation

The system uses deterministic aggregation via Pandas before passing results to LLM.

2. What specific entities or parameters are you extracting from the text?

Your current entity extraction includes:

```
{
   "region": str or None,
   "category": str or None,
   "product": str or None,
   "products": list[str],  # multi-product support
   "metric": str or None,
   "year": int or None,
   "quarter": int or None,
   "time_reference": str or None
}
```

Supported Entities

Entity	Example
product	Laptop
products	[Laptop, Mobile]
category	Electronics
region	South

Entity	Example
metric	sales / revenue / profit
year	2023
quarter	1 (converted internally to "Q1")
time_reference	last_year / this_year

This makes the assistant fully dataset-aware and dynamic.

3. How do you handle edge cases like misspellings or ambiguous text?

Currently, the system handles edge cases using:

a) Lowercase Normalization

Ensures case-insensitive matching.

b) Partial String Matching

Example:

electronics category

Matches “Electronics”

c) Multi-product fallback

If more than one product is detected → switch to comparison mode.

d) Confidence-based fallback

If intent confidence < threshold (e.g., 0.6):
→ System switches to RAG mode.

4. What does your output data structure look like?

Before sending to LLM, the system produces structured output like:

Single Metric Example

```
{
   "result": 45116454,
   "metric": "sales_amount"
}
```

Comparison Example

```
{
```

```
  "comparison": {
    "Laptop": 45116454,
    "Mobile": 52941876
  },
  "metric": "sales_amount"
}
```


Trend Example


```
{
  "trend_data": {
    "Q1": 45116454,
    "Q2": 47892231,
    "Q3": 50122983
  },
  "metric": "sales_amount"
}
```


This structured data is then passed to the LLM as context.

The LLM is only responsible for:

- Formatting
- Explanation
- Suggestion

It does NOT compute numbers.

5. Individual Contribution

- **Designing the Full Pipeline (Jethin Jangiti)**
  - ASR → Intent → NER → Analytics → Visualization → LLM → TTS
- **Building Dataset-Aware NER (Akshaya Alampally)**
  - Dynamic entity extraction from a real dataset
  - Multi-product detection
  - Quarter formatting fix (Q1 string bug)

- **Implementing Structured Analytics Engine (Deepak Manthena)**
  - Pandas-based aggregation
  - Multi-product comparison logic
  - Trend and prediction routing
- **Developing Evaluation System (Chandra Sena Duggirala)**
  - Intent Accuracy measurement
  - Data Retrieval Success Rate
  - Full Pipeline Score
- **Debugging & Optimization**
  - Fixed quarter string mismatch issue
  - Eliminated LLM numeric hallucination
  - Improved confidence routine

System Performance Results

Latest evaluation:

Intent Accuracy: 0.80

Data Retrieval Success Rate: 1.00

Overall System Score: 0.90

This shows:

- Strong deterministic analytics layer
- Reliable entity extraction
- Minor room for improvement in classification